

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: Задача минимизации задержек

Студент гр. 4342	_____	Е.Н. Озернов
Студент гр. 4342	_____	М.Н. Митюков
Студент гр. 4342	_____	Г.А. Алюкин
Руководитель	_____	Т.Р. Жангиров

Санкт-Петербург
2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Е.Н. Озернов

Студент: М.Н. Митюков

Студент: Г.А. Алюкин

Группа: 4342

Тема практики: Задача минимизации задержек

Задание на практику:

Дано N задач, каждая из которых имеет свое время выполнения и срок сдачи к которому она должна быть выполнена. Задача, это составить расписание с минимальными задержками. Задержка – количество времени на которое выполнение задач превысило срок сдачи.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 14.07.2026

Дата защиты отчета: 17.07.2026

Студент гр. 4342

Е.Н. Озернов

Студент гр. 4342

М.Н. Митюков

Студент гр. 4342

Г.А. Алюкин

Руководитель

Т.Р. Жангиров

АННОТАЦИЯ

Отчёт посвящён разработке приложения с графическим интерфейсом для решения задачи минимизации суммарной задержки при планировании работ с помощью генетического алгоритма. Описана доменная модель приложения: представление входных задач (время выполнения и срок сдачи), настраиваемые параметры алгоритма, а также структура состояний популяции, сохраняемых по каждому поколению, изложена реализация генетического алгоритма: кодирование решений в виде перестановок задач, операторы селекции (турнирный отбор), скрещивания (Order Crossover) и мутации (swap mutation), формирование функции приспособленности и механизм ранней остановки, архитектура GUI с пошаговой визуализацией поиска решения и графиком сходимости, а также результаты тестирования программы.

SUMMARY

The report is devoted to the development of an application with a graphical interface to solve the problem of minimizing the total delay in work planning using a genetic algorithm. The domain model of the application is described: the representation of input tasks (execution time and deadline), configurable algorithm parameters, as well as the structure of population states stored for each generation, the implementation of the genetic algorithm is described: encoding solutions in the form of permutations of tasks, selection operators (tournament selection), crosses (Order Crossover) and mutations (swap mutation), the formation of the fitness function and the mechanism of early stopping, GUI architecture with step-by-step visualization of the solution search and convergence graph, as well as the results of testing the program.

СОДЕРЖАНИЕ

Оглавление

ВВЕДЕНИЕ.....	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область	6
1.2 Задачи практики	7
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	9
2.1. Изучение постановки задачи минимизации суммарной задержки	9
2.2. Изучение принципов работы генетических алгоритмов.....	10
2.3. Спроектировать структуру данных для задач, параметров алгоритма и состояний популяции по поколениям	11
2.4. Реализовать генетический алгоритм самостоятельно, с настраиваемыми пользователем параметрами.....	12
2.5. Разработка графического интерфейса с вводом данных вручную, из файла и случайной генерацией	14
2.6. Реализация пошаговой визуализации процесса поиска решения	15
2.7. Реализация графика изменения качества решения по поколениям	15
2.8. Тестирование алгоритма и программы на разных наборах данных	16
2.9. Тестирование графического интерфейса программы	17
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	21
4 ОТЗЫВ РУКОВОДИТЕЛЯ	23
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26

ВВЕДЕНИЕ

Предметная область практики. Практика посвящена разработке программного обеспечения для решения задач теории расписаний с применением генетических алгоритмов. Рассматривается задача минимизации задержек при выполнении набора работ с заданными временами выполнения и сроками сдачи — классическая NP-трудная задача комбинаторной оптимизации, для решения которой применяются приближённые методы, в том числе генетические алгоритмы.

Цель практики. Целью практики является разработка программного приложения с графическим пользовательским интерфейсом, реализующего самостоятельно написанный эволюционный алгоритм для решения задачи минимизации задержек, а также обеспечивающего наглядную пошаговую визуализацию процесса поиска оптимального решения на протяжении работы алгоритма.

Задачи практики. Изучить постановку задачи минимизации задержек и принципы построения эволюционных алгоритмов; разработать графический интерфейс с вводом данных вручную, из файла и случайной генерацией; реализовать эволюционный алгоритм самостоятельно, с настраиваемыми параметрами; реализовать пошаговую визуализацию процесса поиска решения и график изменения функции качества по поколениям; протестировать программу.

1 ПОСТАНОВКА ЗАДАЧИ

Практика выполнялась в составе учебной бригады с распределением ролей между участниками (разработка алгоритмического ядра, разработка графического интерфейса, тестирование и оформление отчётной документации). Ниже описаны предметная область, в рамках которой выполнялась практика, и конкретные задачи, поставленные перед бригадой.

1.1 Предметная область

Задачи составления расписаний занимают важное место в исследовании операций и прикладной информатике. На практике они возникают везде, где нужно распределить во времени набор работ с известной длительностью и сроками завершения. Такие задачи нужны, чтобы планировать производство, распределять нагрузку на серверы, организовывать доставку в логистике и управлять проектами с жесткими сроками сдачи.

Одной из постановок в этой области является задача минимизации задержек: требуется найти такой порядок выполнения работ, при котором суммарное (или максимальное) превышение установленных сроков сдачи будет минимальным. В общем случае данная задача относится к классу NP-трудных, из-за чего точные методы решения (полный перебор, динамическое программирование) становятся неприменимыми при большом числе работ. Поэтому на практике для её решения широко применяются приближённые эвристические и методы, среди которых заметное место занимают генетические алгоритмы — методы оптимизации, имитирующие процессы естественного отбора: формирование популяции решений, их отбор, скрещивание и мутацию с постепенным улучшением качества решений от поколения к поколению.

Генетические алгоритмы – это семейство поисковых алгоритмов, идеи которых подсказаны принципами эволюции в природе. Имитируя процессы естественного отбора и воспроизводства, генетические алгоритмы могут находить высококачественные решения задач, включающих поиск, оптимизацию и обучение. В то же время аналогия с естественным отбором

позволяет этим алгоритмам преодолевать проблему локальных экстремумов, возникающую на пути традиционных алгоритмов поиска и оптимизации, особенно в задачах с большим числом параметров и сложными математическими представлениями. [1]

Помимо алгоритма оптимизации, важной практической задачей является разработка удобного программного инструмента с графическим интерфейсом, позволяющего задавать исходные данные, настраивать параметры алгоритма и, что особенно важно в учебных и демонстрационных целях, визуально наблюдать процесс поиска решения. Подобные инструменты полезны как для изучения самих алгоритмов оптимизации (демонстрация работы эволюционных методов в учебном процессе), так и для практического подбора параметров алгоритма под конкретную задачу.

1.2 Задачи практики

В рамках практики были поставлены следующие задачи:

1. Изучить постановку задачи минимизации суммарной задержки и подходы к её решению;
2. Изучить принципы работы генетических алгоритмов (кодирование решений, селекция, скрещивание, мутация, функция приспособленности);
3. Спроектировать структуру данных для задач, параметров алгоритма и состояний популяции по поколениям;
4. Реализовать генетический алгоритм самостоятельно, с настраиваемыми пользователем параметрами;
5. Разработать графический интерфейс с вводом данных вручную, из файла и случайной генерацией;
6. Реализовать пошаговую визуализацию процесса поиска решения с возможностью просмотра разных решений в популяции, пропуска шагов и отката назад;
7. Реализовать график изменения качества решения по поколениям;
8. Тестирование алгоритма и программы на разных наборах данных;

9. Тестирование графического интерфейса программы.

2 РЕЗУЛЬТАТЫ РАБОТЫ

Распределение задач практики:

Общие задачи (выполнялись всеми участниками):

1. Изучить постановку задачи минимизации суммарной задержки и подходы к её решению;
2. Изучить принципы работы генетических алгоритмов (кодирование решений, селекция, скрещивание, мутация, функция приспособленности);

Е.Н. Озернов:

3. Спроектировать структуру данных для задач, параметров алгоритма и состояний популяции по поколениям;
4. Реализовать генетический алгоритм самостоятельно, с настраиваемыми пользователем параметрами;

М.Н. Митюков:

5. Разработать графический интерфейс с вводом данных вручную, из файла и случайной генерацией;
6. Реализовать пошаговую визуализацию процесса поиска решения с возможностью просмотра разных решений в популяции, пропуска шагов и отката назад;

7. Реализовать график изменения качества решения по поколениям;

Г.А. Алюкин:

8. Тестирование алгоритма и программы на разных наборах данных;
9. Тестирование графического интерфейса программы.

2.1. Изучение постановки задачи минимизации суммарной задержки

Была изучена математическая постановка задачи однопроцессорного планирования работ с целью минимизации суммарной задержки. Рассмотрена формальная модель: набор из N работ, каждая из которых характеризуется временем выполнения t_i и сроком сдачи d_i . Работы выполняются последовательно, без прерываний, на одном исполнителе. Введено понятие задержки отдельной работы как $T_i = \max(0, C_i - d_i)$, где C_i — момент

завершения работы при заданном порядке выполнения, а также целевая функция — суммарная задержка $\sum_i T_i$, подлежащая минимизации.

Изучены существующие подходы к решению подобных задач: точные методы (полный перебор, динамическое программирование), применимые лишь при небольшой размерности задачи, и приближённые методы. Установлено, что задача минимизации суммарной задержки в общем случае является NP-трудной, в отличие от задачи минимизации максимальной задержки, для которой существует точный полиномиальный алгоритм (правило EDD). Это обусловило выбор приближённого эволюционного метода в качестве основного инструмента решения.

2.2. Изучение принципов работы генетических алгоритмов

Были изучены общие принципы построения генетических алгоритмов как метода приближённой оптимизации, основанного на имитации процессов естественного отбора. Рассмотрены следующие основные понятия и компоненты:

1. Кодирование решений — представление кандидатного решения (особи) в виде хромосомы; для задачи планирования работ рассмотрено представление решения в виде перестановки индексов работ;
2. Функция приспособленности (fitness) — способ оценки качества решения, в данном случае — значение суммарной задержки, соответствующей порядку работ, заданному особью;
3. Селекция — способы отбора родительских особей для формирования следующего поколения, в частности изучена турнирная селекция;
4. Скрещивание (crossover) — операторы формирования потомков на основе родительских особей; для решений, представленных перестановками, изучен оператор Order Crossover (OX), сохраняющий корректность перестановки у потомка;
5. Мутация — оператор случайного изменения особи для поддержания разнообразия популяции и предотвращения преждевременной

сходимости; рассмотрен оператор перестановки двух случайных элементов (swap mutation).

Также изучены вспомогательные механизмы, применяемые в генетических алгоритмах: элитизм (сохранение наиболее приспособленных особей без изменений при переходе к новому поколению) и ранняя остановка алгоритма при отсутствии улучшения решения в течение заданного числа поколений (stagnation limit).

2.3. Спроектировать структуру данных для задач, параметров алгоритма и состояний популяции по поколениям

Для реализации генетического алгоритма была спроектирована отдельная структура данных, обеспечивающая хранение входных параметров, промежуточных состояний и итоговых результатов работы алгоритма. Проектирование выполнялось с учетом последующей интеграции с графическим интерфейсом, в котором требуется не только получение окончательного решения, но и пошаговая визуализация процесса эволюции популяции.

Для представления одной задачи была введена структура Task, содержащая идентификатор задачи, длительность выполнения и срок завершения. Такая модель позволяет хранить минимально необходимый набор данных для построения расписания и вычисления суммарной задержки.

Для хранения параметров генетического алгоритма была разработана структура GAConfig. В нее были включены размер популяции, число поколений, вероятность мутации, вероятность скрещивания, число элитных особей, размер турнира для селекции, а также параметры выбора методов селекции, скрещивания и мутации. Дополнительно были предусмотрены поля для задания начального зерна генератора случайных чисел, ограничения по стагнации и режима сохранения истории. Такое решение позволило отделить конфигурацию алгоритма от его логики и обеспечить удобное изменение параметров без переработки основного кода.

Для представления одной особи популяции была использована структура `Individual`, содержащая хромосому в виде перестановки индексов задач и кэшированное значение приспособленности. В качестве хромосомы была выбрана именно перестановка, так как задача сводится к поиску оптимального порядка выполнения работ на одном процессоре.

Для хранения информации о конкретном решении в форме, удобной для визуализации, была введена структура `SolutionSnapshot`. В ней сохраняются порядок задач, времена завершения, значения задержек по каждой задаче и суммарная задержка. Наличие такой структуры позволяет избежать повторных вычислений в интерфейсной части программы.

Для представления состояния алгоритма на каждом поколении была разработана структура `GenerationState`. В ней сохраняются номер поколения, вся текущая популяция, эффективное решение в поколении, глобально оптимальное решение на текущий момент, минимальное значение функции, среднее значение по популяции, массив значений приспособленности и история изменения максимального результата. Такая организация данных обеспечивает возможность пошагового отображения процесса работы алгоритма, построения графиков и возврата к любому предыдущему поколению.

В результате была получена согласованная система структур данных, охватывающая все основные сущности задачи: входные данные, параметры алгоритма, отдельные решения и историю эволюции популяции. Спроектированная модель стала основой для последующей реализации генетического алгоритма и его интеграции с графическим интерфейсом.

2.4. Реализовать генетический алгоритм самостоятельно, с настраиваемыми пользователем параметрами

Для решения задачи минимизации суммарной задержки был реализован собственный генетический алгоритм без использования готовых специализированных библиотек. Реализация была выполнена в виде отдельного программного модуля, содержащего всю основную логику

формирования популяции, оценки решений, отбора, скрещивания, мутации и перехода между поколениями.

В качестве представления решения была выбрана хромосома в виде перестановки задач. Такой подход соответствует постановке задачи однопроцессорного расписания, в которой требуется определить оптимальный порядок выполнения работ. Для каждой особи вычислялись времена завершения задач, индивидуальные значения задержек и суммарная задержка, используемая в качестве целевой функции. Чем меньше данное значение, тем более качественным считается решение.

Работа алгоритма начиналась с генерации начальной популяции, состоящей из случайных перестановок всех задач. После этого для каждой особи рассчитывалось значение приспособленности, и формировалось начальное состояние популяции. Далее на каждом поколении выполнялся переход к новой популяции. Сначала без изменений переносилась элитная часть наиболее эффективных решений, что позволяло сохранять уже найденные удачные варианты. Остальная часть популяции формировалась за счет селекции родительских особей, применения оператора скрещивания и последующей мутации потомков.

В качестве базового метода отбора была реализована турнирная селекция. Ее использование позволило обеспечить достаточно простой и эффективный механизм выбора более качественных особей для размножения. Для скрещивания перестановок был реализован оператор Order Crossover, корректно сохраняющий структуру перестановки без повторений и потерь элементов. В качестве стандартной мутации была выбрана swap mutation, при которой две случайные задачи меняются местами. Дополнительно были реализованы расширения в виде PMX-скрещивания и inversion mutation, что позволило сделать алгоритм более гибким и пригодным для дальнейших экспериментов с параметрами.

Все основные характеристики алгоритма были вынесены в настраиваемую конфигурацию. Пользователь может задавать размер

популяции, число поколений, вероятность мутации, вероятность скрещивания, число элитных особей, размер турнира, используемые методы скрещивания и мутации, начальное значение генератора случайных чисел и ограничение на число поколений без улучшения результата. Благодаря этому реализация не является жестко фиксированной и может использоваться для исследования влияния параметров на качество получаемых решений.

Дополнительно в алгоритм был включен механизм ранней остановки. Если в течение заданного числа поколений не происходило улучшения оптимального найденного результата, выполнение завершалось досрочно. Это позволило сократить лишние вычисления в случаях, когда популяция переставала демонстрировать прогресс.

В результате был реализован самостоятельный генетический алгоритм, полностью соответствующий поставленной задаче и допускающий настройку ключевых параметров пользователем. Созданная реализация обеспечила как получение итогового решения, так и основу для последующего анализа поведения алгоритма в процессе его работы.

2.5. Разработка графического интерфейса с вводом данных вручную, из файла и случайной генерацией

Разработан графический интерфейс приложения, реализующий все три предусмотренных способа ввода исходных данных. На вкладке «Источник данных» размещён блок выбора источника с тремя вариантами: ввод вручную, загрузка из файла и случайная генерация. При выборе варианта отображается соответствующая панель: для ручного ввода — подсказка по редактированию таблицы задач, для загрузки из файла — кнопка выбора файла и поле пути, для случайной генерации — поля параметров генерации.

Под блоком выбора источника размещена таблица задач с колонками «№», «Время выполнения» и «Срок сдачи», доступная для ручного редактирования значений. Управление содержимым таблицы обеспечивается кнопками «Добавить задачу», «Удалить выбранную» и «Очистить таблицу».

Графический интерфейс поддерживает весь запланированный функционал: элементы ввода и переключения источников данных отображаются и реагируют на действия пользователя, корректно выполняется обработка данных, введенные значения сохраняются и передаются в алгоритмическую часть приложения.

2.6. Реализация пошаговой визуализации процесса поиска решения

Для отображения хода работы генетического алгоритма разработана вкладка «Запуск», содержащая необходимые элементы визуализации. В верхней части вкладки размещены список особей текущего поколения, позволяющий выбирать и просматривать отдельные особи, а также блок «Текущее расписание» с областью диаграммы Ганта и кнопками управления масштабом отображения (“–“, «По ширине», “+”).

В нижней части вкладки размещён блок «Информация» с полями “Текущее поколение”, “Максимальное значение Fitness”, “Среднее Fitness”, “Минимальное Fitness”, “Суммарная задержка” и “Оптимальное расписание”, предназначенными для отображения текущего состояния поиска решения.

Управление процессом визуализации реализовано через набор кнопок: “Старт”, “Следующий шаг”, “Предыдущий шаг”, “До конца” и “Сброс”. Такой набор элементов управления обеспечивает как пошаговый просмотр эволюции популяции, так и переход к финальному решению без пошагового отображения, а также возврат на предыдущие шаги истории поколений.

Диаграмма Ганта, отображающая расписание задач выбранной особи в рамках текущего поколения, вынесена в отдельный виджет (gantt_chart_view.py).

2.7. Реализация графика изменения качества решения по поколениям

Для отображения динамики изменения функции качества решения в процессе работы алгоритма разработан отдельный виджет графика (fitness_plot.py) на основе библиотеки Matplotlib, размещённый в блоке “График функции качества” на вкладке “Запуск”. Виджет подготовлен для отображения зависимости значения функции качества (суммарной задержки)

от номера поколения с возможностью пополнения графика новыми точками по мере поступления данных от алгоритма.

На графике отображаются линии, соответствующие максимальному, среднему и минимальному значению Fitness по поколениям.

2.8. Тестирование алгоритма и программы на разных наборах данных

После реализации генетического алгоритма было выполнено тестирование программы на различных наборах входных данных. Целью данного этапа являлась проверка корректности вычислений, устойчивости работы алгоритма на типовых и граничных случаях, а также подтверждение готовности программного модуля к дальнейшей интеграции с графическим интерфейсом.

Было проведено модульное (Unit) и интеграционное тестирование с использованием фреймворка `pytest`. Проверки охватывают расчеты фитнес-функции, операторы скрещивания/мутации и логику управления жизненным циклом популяции. Приведены результаты тестирования программы `test_genetic_scheduler.py` (таблица 1).

Таблица 1 – Тестирование программной логики

№	Объект тестирования	Тест	Результат
1	Фитнес-функция	Расчет суммарной задержки для ручного примера (3 задачи).	Passed
2	Операторы GA	Проверка сохранения корректности перестановок после OX, PMX, Swap и Inversion.	Passed
3	Детерминизм	Воспроизводимость результатов при фиксированном random_seed.	Passed
4	Условия остановки	Срабатывание лимита стагнации при отсутствии улучшений в N поколениях.	Passed
5	Граничные случаи	Работа алгоритма с одной задачей и в ситуации «нулевой» задержки.	Passed
6	Управление памятью	Корректное отключение сохранения истории поколений для экономии ресурсов.	Passed

2.9. Тестирование графического интерфейса программы

Для обеспечения надежности интерфейса было проведено:

Автоматизированное тестирование: С использованием фреймворка pytest и библиотеки pytest-qt. Эти тесты имитируют низкоуровневые события (нажатия клавиш, клики мыши) и проверяют внутреннее состояние объектов (свойства isEnabled, text, rowCount). Тестирование проведено в исполняемом файле test_gui.py [2].

Ручное тестирование: Проведено по разработанным тест-кейсам (таблица 2) для проверки визуального отклика и корректности отображения графиков. Приведены примеры работающего интерфейса программы (см. рис. 1, рис. 2, рис. 3).

Таблица 2 – Ручное тестирование GUI

№	Сценарий	Тест	Результат
1	Загрузка данных из файла	1. Вкладка "Источник данных" -> "Загрузить из файла" 2. Выбрать файл test.txt 3. Нажать "Обновить"	Таблица заполняется данными. Внизу надпись "Загружено задач: N". Кнопка "Старт" активна
2	Смена алгоритма (OX/PMX)	1. Вкладка "Параметры" 2. Изменить тип скрещивания на PMX. 3. Запустить алгоритм.	Алгоритм успешно завершается, используя выбранный метод.
3	Визуализация (Gantt Zoom)	1. Запустить алгоритм. 2. На вкладке "Запуск" нажать + на панели графика Ганта.	Масштаб диаграммы увеличивается, появляются полосы прокрутки при необходимости.
4	Пошаговая отладка	1. Запустить алгоритм. 2. Нажать "Предыдущий шаг" (если не 0 поколение). 3. Нажать "Следующий шаг".	Текст "Текущее поколение" меняется. График Fitness перерисовывается.

Решение задачи минимизации суммарной задержки генетическим алгоритмом

Источник данных Параметры алгоритма Запуск

Источник данных

☐ Ввод вручную ☐ Загрузить из файла ☒ Случайная генерация

Количество задач 12

Минимальное время выполнения 1

Максимальное время выполнения 12

Минимальный дедлайн 8

Максимальный дедлайн 60

Сгенерировать

Таблица задач

№	Время выполнения	Дедлайн
1	11	15
2	1	55
3	5	23
4	4	16
5	12	14
6	11	55
7	9	13
8	10	35
9	1	9
10	2	21
11	4	40

Расчет завершен: выполнено заданное число поколений

Добавить задачу Удалить выбранную Очистить таблицу

Всего задач: 12

Рисунок 1 — Вкладка «Источник данных»

Решение задачи минимизации суммарной задержки генетическим алгоритмом

Источник данных Параметры алгоритма Запуск

Параметры генетического алгоритма

Размер популяции 80 Тип отбора Турнирный

Количество поколений 50 Тип скрещивания OX

Вероятность скрещивания 0.85 Тип мутации Swap

Вероятность мутации 0.12 Элитизм ☒ Использовать

Количество лучших особей 1

Размер турнира 3

Лимит стагнации Отключено

Seed 42

Рисунок 2 — Вкладка «Параметры алгоритма»

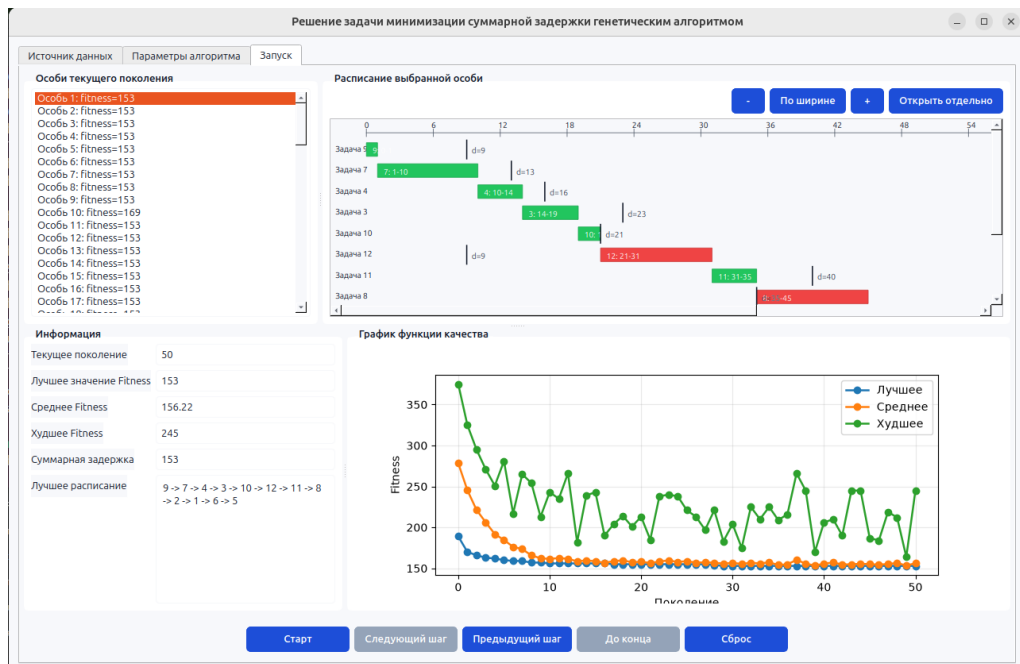


Рисунок 3 — Вкладка «Запуск»

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В качестве основного языка разработки был взят Python 3. Данный выбор обоснован наличием развитой стандартной библиотеки и широким применением данного языка для реализации алгоритмов оптимизации и комбинаторных задач. Для проектирования структур данных активно использовался модуль `dataclasses`, что позволило формализовать параметры генетического алгоритма и состояния популяции. Для повышения надежности кода и упрощения процесса совместной разработки применялось аннотирование типов с использованием модуля `typing`. Логика вероятностного поиска в алгоритме (формирование начальной популяции, селекция, скрещивание и мутация) была реализована с применением модуля `random`.

Построение графического пользовательского интерфейса (GUI) осуществлялось с использованием фреймворка `PyQt6`. Средствами данной библиотеки были реализованы основные компоненты системы: главное окно приложения, интерфейс со вкладками, интерактивные таблицы для ввода данных и формы управления параметрами эволюции. На базе графических примитивов `PyQt6` был разработан специализированный виджет для визуализации диаграммы Ганта, обеспечивающий наглядное отображение расписания и моментов завершения задач.

Для реализации инструментов анализа и визуализации была интегрирована библиотека `Matplotlib`. На её основе разработан виджет графика, поддерживающий обновление данных в реальном времени. Это позволило обеспечить визуализацию изменения функции качества (Fitness) на протяжении всех поколений, отражая процесс сходимости алгоритма к оптимальному решению.

Для проведения автоматизированного тестирования использовался фреймворк `pytest`. С его помощью была организована проверка математической корректности алгоритма и обработки граничных случаев. Тестирование графического интерфейса проводилось с использованием

расширения `pytest-qt`, что позволило автоматизировать проверку работы элементов управления и механизмов валидации пользовательского ввода.

Среда разработки включала использование интерпретатора Python 3 и интерфейса командной строки для управления процессом запуска и отладки.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно
добавить формулировку:

По результатам работы учебная практика оценена на <ваша оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. 4).

Отзыв

о прохождении учебной практики

«ФИО студента», студент(ка) группы «номер группы», второго
курса бакалавриата Санкт-Петербургского электротехнического
университета «ЛЭТИ» им В.И. Ульянова, проходил(а) учебную практику
по теме ««тема практики»» с «дата начала» по «дата окончания».

В течение практики обучающийся(аяся) <ФИО студента> выполнял(а) <задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок 4 — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

1. Изучить постановку задачи: Проведен анализ модели однопроцессорного планирования. Обосновано применение эвристических методов (ГА) для решения NP-трудной задачи минимизации задержки.
2. Изучить принципы генетических алгоритмов: Изучены механизмы эволюционного поиска. Выбраны операторы (ОХ, РМХ, мутации), сохраняющие целостность расписания при скрещивании
3. Проектирование структур данных: Спроектированы классы (Task, GAConfig, GenerationState), обеспечившие разделение вычислительной логики и интерфейса, а также хранение истории эволюции.
4. Реализация алгоритма: На языке Python реализован кастомизируемый генетический алгоритм с возможностью настройки параметров селекции, скрещивания, мутации и элитизма.
5. Разработка интерфейса: На базе PyQt6 создан GUI с поддержкой ручного ввода данных, импорта из файлов и автоматической генерации тестовых наборов.
6. Пошаговая визуализация: Реализована система навигации по поколениям с возможностью просмотра промежуточных решений на динамической диаграмме Ганта.
7. Графический анализ: Интегрирован модуль Matplotlib для визуализации динамики Fitness-функции, наглядно подтверждающий сходимость алгоритма к оптимуму.
8. Тестирование алгоритма: Проведено тестирование на различных данных; подтверждена стабильность формирования истории и корректность генетических преобразований.
9. Тестирование интерфейса: Выполнено комплексное тестирование (11 сценариев). Подтверждена корректность валидации ввода и отказоустойчивость системы при некорректных данных.

Общий результат: Цель работы достигнута. Созданная программа представляет собой законченное решение, объединяющее в себе гибкий вычислительный инструмент и систему визуализации процессов эволюционного поиска. Разработанное приложение может быть использовано как для решения практических задач планирования, так и в учебных целях для демонстрации работы стохастических алгоритмов оптимизации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Вирсански, Э. Генетические алгоритмы на Python / Э. Вирсански ; пер. с англ. А. А. Слинкина. – М. : ДМК Пресс, 2020. – 286 с
2. Код проекта (GitHub): <https://github.com/EvgenyOzernov/EdProject>